

Laboratorio 02 – Implementar objetos de programabilidad con SQL

Entorno: SQL Server + SSMS · Base de datos de ejemplo **AdventureWorksLT2025** (esquema **SalesLT**)

Enunciado: [Implement programmability objects with SQL – Microsoft Learn](#)

Objetivo

Centralizar lógica dentro de la base de datos para mejorar la mantenibilidad, creando y probando los objetos de programabilidad principales de SQL Server:

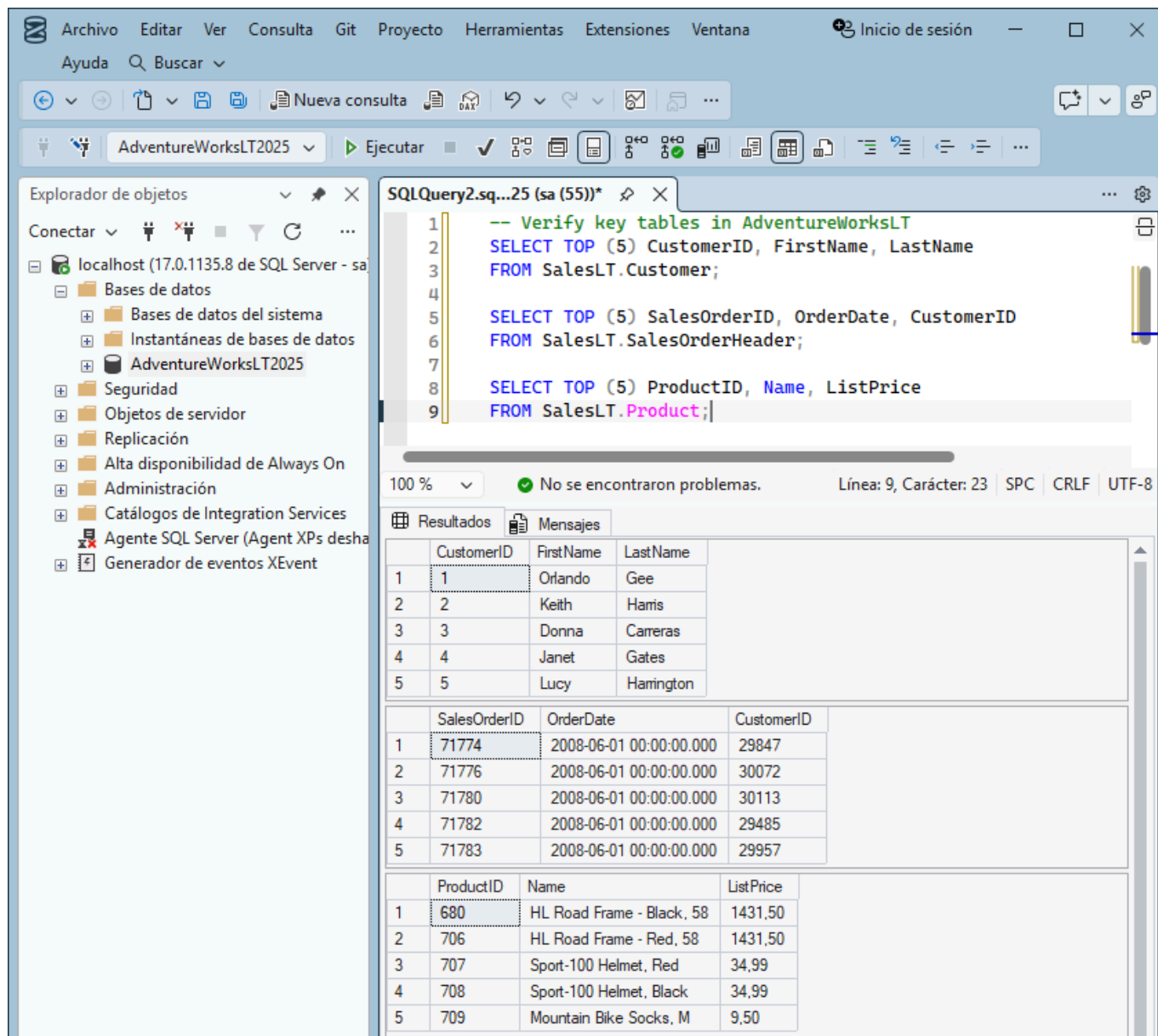
Objeto	Nombre	Para qué sirve
Vista	<code>SalesLT.vCustomerOrders</code>	Ocultar la complejidad de un JOIN
Procedimiento almacenado	<code>dbo.AddOrderLineItem</code>	Encapsular una operación de negocio transaccional
Función escalar	<code>dbo.fnOrderTotal</code>	Cálculo reutilizable que devuelve un único valor
Función con valores de tabla (TVF)	<code>dbo.GetCustomerOrders</code>	Devolver un conjunto de filas parametrizado
Trigger	<code>SalesLT.trg_LogOrderTotalChange</code>	Reaccionar automáticamente a cambios (auditoría)

Los apartados se han ejecutado en este mismo orden sobre una base de datos recién restaurada, porque el orden importa: cada objeto se apoya en los anteriores y el trigger solo audita lo que ocurre **después** de crearlo.

1. Conexión y verificación de AdventureWorksLT2025

Antes de crear nada compruebo que la base de datos está restaurada y que tengo acceso de lectura a las tablas que voy a usar.

```
SELECT TOP (5) CustomerID, FirstName, LastName FROM SalesLT.Customer;  
SELECT TOP (5) SalesOrderID, OrderDate, CustomerID FROM SalesLT.SalesOrderHeader;  
SELECT TOP (5) ProductID, Name, ListPrice FROM SalesLT.Product;
```



The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Explorador de objetos' (Object Explorer) pane displays the server hierarchy for 'localhost (17.0.1135.8 de SQL Server - sa)'. The 'AdventureWorksLT2025' database is selected. The main pane shows a SQL query titled 'SQLQuery2.sql...25 (sa (55))*'. The query is as follows:

```
-- Verify key tables in AdventureWorksLT
SELECT TOP (5) CustomerID, FirstName, LastName
FROM SalesLT.Customer;

SELECT TOP (5) SalesOrderID, OrderDate, CustomerID
FROM SalesLT.SalesOrderHeader;

SELECT TOP (5) ProductID, Name, ListPrice
FROM SalesLT.Product;
```

Below the query, the 'Resultados' (Results) pane shows three tables of data. The first table is 'CustomerID, FirstName, LastName' with 5 rows. The second table is 'SalesOrderID, OrderDate, CustomerID' with 5 rows. The third table is 'ProductID, Name, ListPrice' with 5 rows.

CustomerID	FirstName	LastName
1	Orlando	Gee
2	Keith	Harris
3	Donna	Carreras
4	Janet	Gates
5	Lucy	Hamington

SalesOrderID	OrderDate	CustomerID
1	2008-06-01 00:00:00.000	29847
2	2008-06-01 00:00:00.000	30072
3	2008-06-01 00:00:00.000	30113
4	2008-06-01 00:00:00.000	29485
5	2008-06-01 00:00:00.000	29957

ProductID	Name	ListPrice
1	HL Road Frame - Black, 58	1431,50
2	HL Road Frame - Red, 58	1431,50
3	Sport-100 Helmet, Red	34,99
4	Sport-100 Helmet, Black	34,99
5	Mountain Bike Socks, M	9,50

Resultado: se obtienen tres conjuntos de resultados (clientes, pedidos y productos) de hasta 5 filas cada uno, lo que confirma que la conexión y la base de datos funcionan correctamente.

2. Vista para simplificar consultas

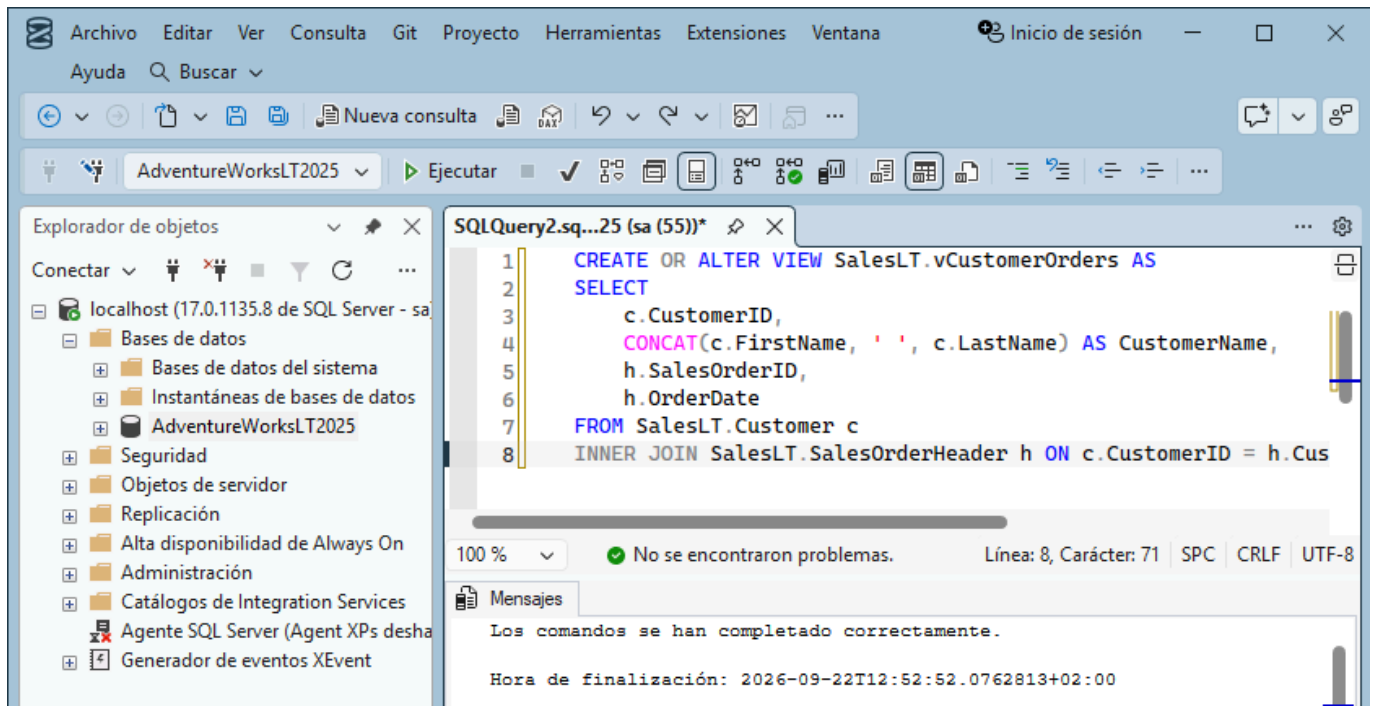
2.1 Crear la vista

La vista une clientes y cabeceras de pedido. La aplicación ya no necesita escribir el **JOIN**: consulta la vista como si fuera una tabla.

```
CREATE OR ALTER VIEW SalesLT.vCustomerOrders AS
SELECT
    c.CustomerID,
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    h.SalesOrderID,
    h.OrderDate
```

```
FROM SalesLT.Customer c
INNER JOIN SalesLT.SalesOrderHeader h ON c.CustomerID = h.CustomerID;
```

- **CREATE OR ALTER** crea la vista o la sobrescribe si ya existe (permite relanzar el script sin errores).
- **CONCAT** genera el nombre completo del cliente en una sola columna.
- **INNER JOIN** solo devuelve clientes que tienen al menos un pedido.



2.2 Validar la vista

```
SELECT TOP (5) * FROM SalesLT.vCustomerOrders ORDER BY OrderDate DESC;
```

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Explorador de objetos' (Object Explorer) pane displays the server structure for 'localhost (17.0.1135.8 de SQL Server - sa)'. The 'AdventureWorksLT2025' database is selected. The main pane shows a query window titled 'SQLQuery2.sql...25 (sa (55))*' with the following SQL code:

```
1 SELECT TOP (5) *
2 FROM SalesLT.vCustomerOrders
3 ORDER BY OrderDate DESC;
```

Below the query window, the 'Resultados' (Results) pane shows the output of the query. The status bar indicates 'No se encontraron problemas.' (No problems found) and 'Línea: 3, Carácter: 26' (Line: 3, Character: 26). The results are displayed in a table with the following columns: CustomerID, CustomerName, SalesOrderID, and OrderDate.

	CustomerID	CustomerName	SalesOrderID	OrderDate
1	29847	David Hodgson	71774	2008-06-01 00:00:00.000
2	30072	Andrea Thomsen	71776	2008-06-01 00:00:00.000
3	30113	Raja Venugopal	71780	2008-06-01 00:00:00.000
4	29485	Catherine Abel	71782	2008-06-01 00:00:00.000
5	29957	Kevin Liu	71783	2008-06-01 00:00:00.000

Resultado: los 5 pedidos más recientes con **CustomerID**, nombre completo, **SalesOrderID** y **OrderDate**, sin haber escrito ningún **JOIN** en la consulta.

3. Procedimiento almacenado para procesar un pedido

3.1 Crear el procedimiento

dbo.AddOrderLineItem añade una línea a un pedido existente y recalcula el subtotal de la cabecera, todo dentro de una **transacción**.

```
CREATE OR ALTER PROCEDURE dbo.AddOrderLineItem
    @SalesOrderID INT,
    @ProductID INT,
    @Quantity INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;

    DECLARE @UnitPrice DECIMAL(18,2);
    SELECT @UnitPrice = CAST(ListPrice AS DECIMAL(18,2))
    FROM SalesLT.Product WHERE ProductID = @ProductID;

    IF @UnitPrice IS NULL
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50010, 'Invalid ProductID specified.', 1;
    END
END
```

```
IF NOT EXISTS (SELECT 1 FROM SalesLT.SalesOrderHeader WHERE SalesOrderID =
@SalesOrderID)
BEGIN
    ROLLBACK TRANSACTION;
    THROW 50011, 'Invalid SalesOrderID specified.', 1;
END

INSERT INTO SalesLT.SalesOrderDetail (SalesOrderID, OrderQty, ProductID,
UnitPrice, UnitPriceDiscount)
VALUES (@SalesOrderID, @Quantity, @ProductID, @UnitPrice, 0);

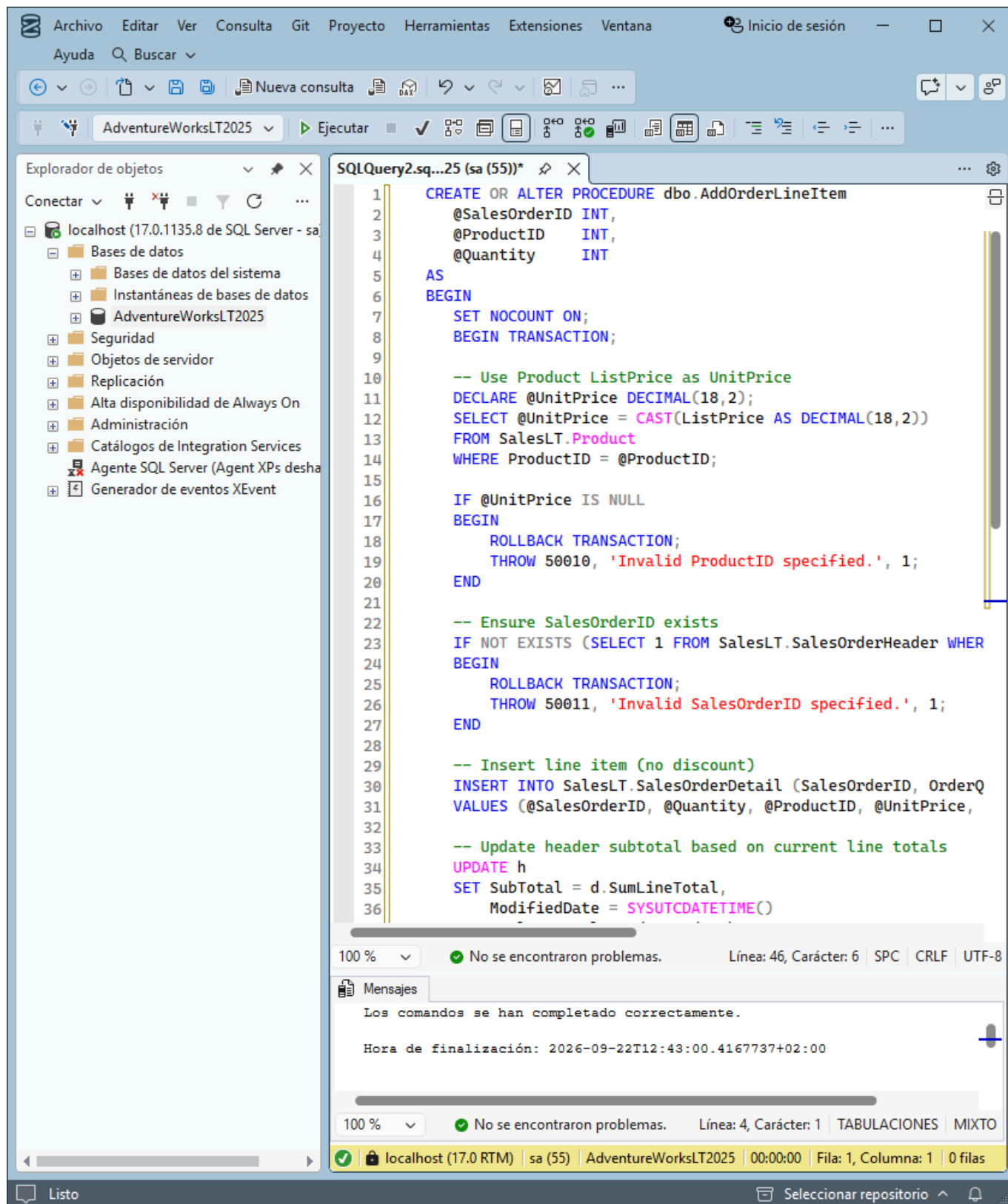
UPDATE h
SET SubTotal = d.SumLineTotal, ModifiedDate = SYSUTCDATETIME()
FROM SalesLT.SalesOrderHeader h
INNER JOIN (
    SELECT SalesOrderID, SUM(LineTotal) AS SumLineTotal
    FROM SalesLT.SalesOrderDetail
    WHERE SalesOrderID = @SalesOrderID
    GROUP BY SalesOrderID
) d ON d.SalesOrderID = h.SalesOrderID;

COMMIT TRANSACTION;
END;
```

Funcionamiento paso a paso:

1. **Validación del producto:** toma su **ListPrice** como precio unitario; si no existe → **ROLLBACK** + error 50010.
2. **Validación del pedido:** si el **SalesOrderID** no existe → **ROLLBACK** + error 50011.
3. **Inserción** de la línea en **SalesOrderDetail** (sin descuento).
4. **Recálculo** del **SubTotal** de la cabecera sumando **LineTotal** de todas sus líneas.
5. **COMMIT:** la operación es **atómica**; o se aplican todos los cambios o ninguno.

SET NOCOUNT ON evita los mensajes "(n rows affected)", que añaden tráfico innecesario.



3.2 Probar el procedimiento

Se usa el último pedido existente y el producto 680, cantidad 1:

```

DECLARE @SalesOrderID INT = (SELECT TOP 1 SalesOrderID
    FROM SalesLT.SalesOrderHeader
    ORDER BY SalesOrderID DESC);

EXEC dbo.AddOrderLineItem @SalesOrderID = @SalesOrderID, @ProductID = 680,

```

```

@Quantity = 1;

SELECT TOP (5) * FROM SalesLT.SalesOrderDetail
WHERE SalesOrderID = @SalesOrderID ORDER BY SalesOrderDetailID DESC;

SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue
FROM SalesLT.SalesOrderHeader WHERE SalesOrderID = @SalesOrderID;

```

The screenshot shows the SQL Server Enterprise Edition interface. The left pane displays the 'Explorador de objetos' (Object Explorer) with the 'AdventureWorksLT2025' database selected. The main pane shows a SQL query in 'SQLQuery2.sql...25 (sa (55))*'. The query is as follows:

```

1  -- Add a line item to an existing order (choose a valid Sales
2  DECLARE @SalesOrderID INT = (SELECT TOP 1 SalesOrderID
3                                FROM SalesLT.SalesOrderHeader
4                                ORDER BY SalesOrderID DESC);
5  EXEC dbo.AddOrderLineItem @SalesOrderID = @SalesOrderID,
6                             @ProductID = 680,
7                             @Quantity = 1; -- adjust ProductID
8
9  SELECT TOP (5) *
10 FROM SalesLT.SalesOrderDetail
11 WHERE SalesOrderID = @SalesOrderID
12 ORDER BY SalesOrderDetailID DESC;
13
14 SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue
15 FROM SalesLT.SalesOrderHeader
16 WHERE SalesOrderID = @SalesOrderID;

```

The status bar indicates 'No se encontraron problemas.' (No problems found). The 'Resultados' (Results) pane shows two tables:

	SalesOrderID	SalesOrderDetailID	OrderQty	ProductID	UnitPrice	UnitPriceDiscount	LineTotal
1	71946	113407	1	680	1431,50	0,00	1431,500
2	71946	113406	1	916	31,584	0,00	31,58400

	SalesOrderID	SubTotal	TaxAmt	Freight	TotalDue
1	71946	1463,084	3,1163	0,9738	1467,1741

Resultado: la nueva línea (ProductID 680, precio 1431,50) aparece la primera en el detalle del pedido 71946, y la cabecera muestra el **SubTotal** actualizado a 1463,084. **TotalDue** también cambia porque es una columna calculada a partir de **SubTotal + TaxAmt + Freight**.

Matiz importante: el procedimiento recalcula el **SubTotal**, pero **no TaxAmt** ni **Freight**, que siguen siendo los del pedido original (3,1163 y 0,9738). Por eso el **TotalDue** resultante es coherente con la fórmula de la columna calculada, pero no con las reglas de negocio reales: para que lo fuera habría que recalcular también impuestos y portes a partir del nuevo subtotal.

4. Función escalar para cálculos reutilizables

4.1 Crear la función

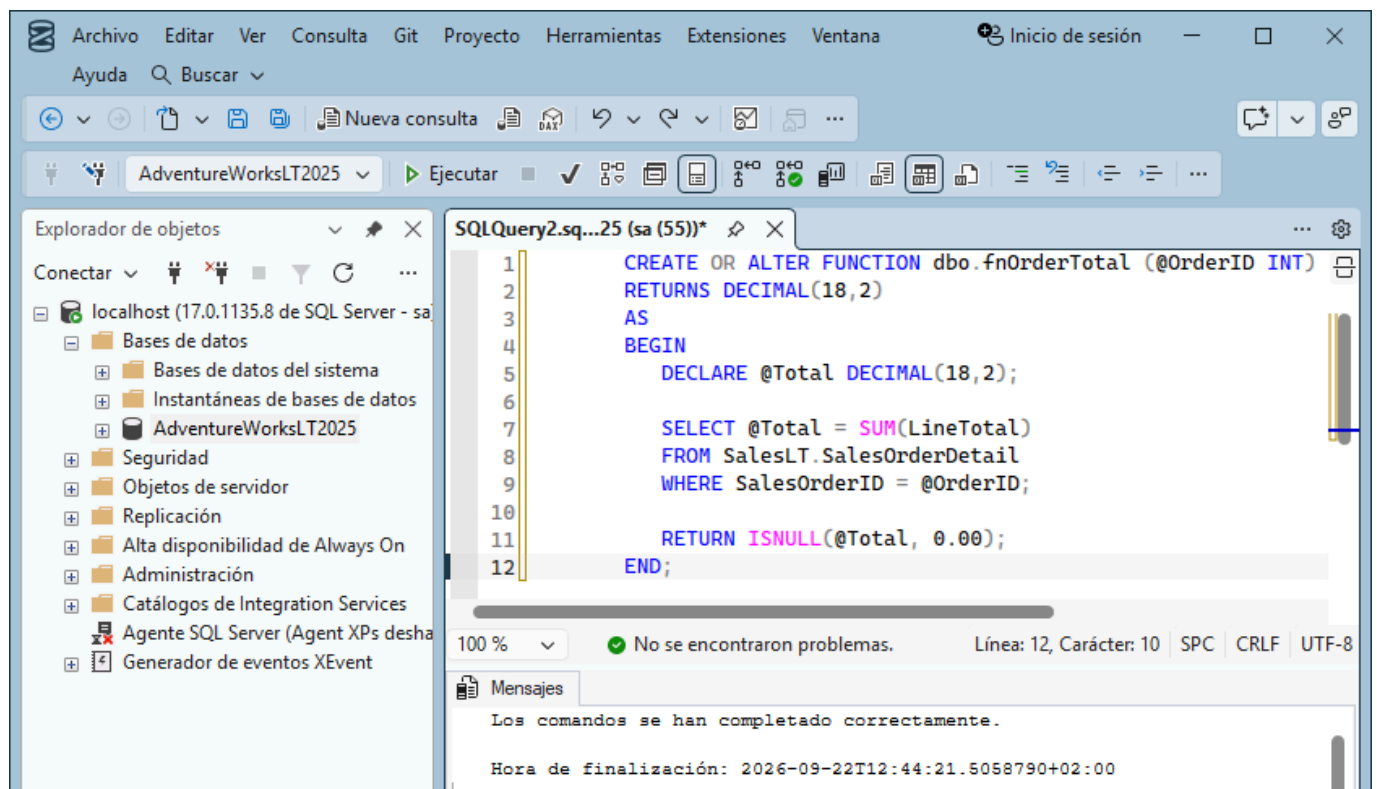
Una función escalar devuelve **un único valor**. **dbo.fnOrderTotal** devuelve el importe total de un pedido.

```
CREATE OR ALTER FUNCTION dbo.fnOrderTotal (@OrderID INT)
RETURNS DECIMAL(18,2)
AS
BEGIN
    DECLARE @Total DECIMAL(18,2);

    SELECT @Total = SUM(LineTotal)
    FROM SalesLT.SalesOrderDetail
    WHERE SalesOrderID = @OrderID;

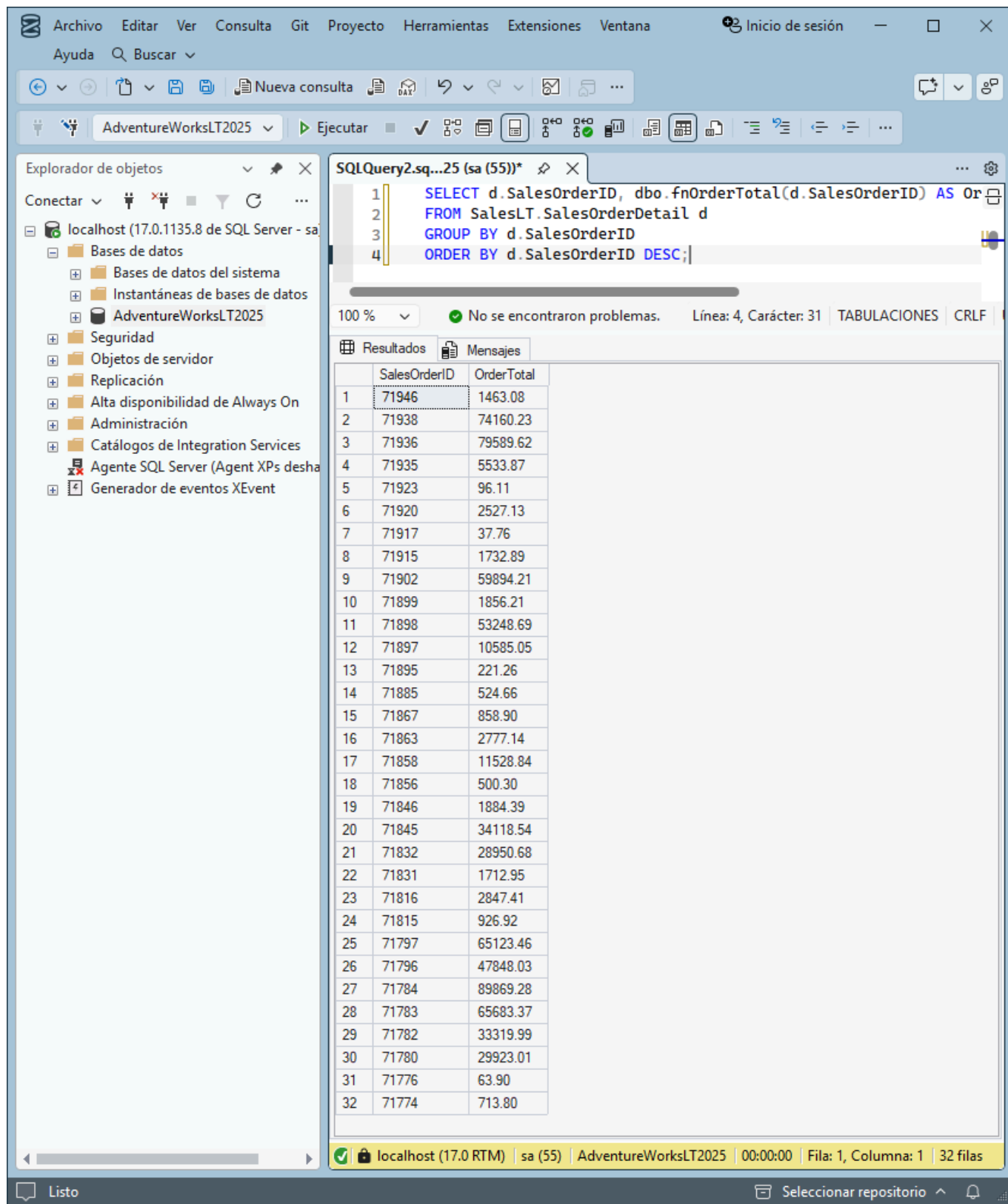
    RETURN ISNULL(@Total, 0.00);
END;
```

`ISNULL` garantiza que un pedido sin líneas devuelva `0.00` en lugar de `NULL`.



4.2 Usar la función

```
SELECT d.SalesOrderID, dbo.fnOrderTotal(d.SalesOrderID) AS OrderTotal
FROM SalesLT.SalesOrderDetail d
GROUP BY d.SalesOrderID
ORDER BY d.SalesOrderID DESC;
```



The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the 'Explorador de objetos' (Object Explorer) with the 'AdventureWorksLT2025' database selected. The right pane shows the 'SQLQuery2.sql...25 (sa (55))' window with the following query:

```
1 SELECT d.SalesOrderID, dbo.fnOrderTotal(d.SalesOrderID) AS OrderTotal
2 FROM SalesLT.SalesOrderDetail d
3 GROUP BY d.SalesOrderID
4 ORDER BY d.SalesOrderID DESC;
```

The query execution status bar indicates 'No se encontraron problemas.' (No problems found). The status bar also shows 'Línea: 4, Carácter: 31' and 'TABULACIONES CRLF'. The 'Resultados' (Results) pane displays the following data:

	SalesOrderID	OrderTotal
1	71946	1463.08
2	71938	74160.23
3	71936	79589.62
4	71935	5533.87
5	71923	96.11
6	71920	2527.13
7	71917	37.76
8	71915	1732.89
9	71902	59894.21
10	71899	1856.21
11	71898	53248.69
12	71897	10585.05
13	71895	221.26
14	71885	524.66
15	71867	858.90
16	71863	2777.14
17	71858	11528.84
18	71856	500.30
19	71846	1884.39
20	71845	34118.54
21	71832	28950.68
22	71831	1712.95
23	71816	2847.41
24	71815	926.92
25	71797	65123.46
26	71796	47848.03
27	71784	89869.28
28	71783	65683.37
29	71782	33319.99
30	71780	29923.01
31	71776	63.90
32	71774	713.80

The status bar at the bottom shows 'localhost (17.0 RTM) | sa (55) | AdventureWorksLT2025 | 00:00:00 | Fila: 1, Columna: 1 | 32 filas'.

Resultado: un total por pedido (32 filas). El pedido 71946 devuelve 1463,08, el mismo importe que el **SubTotal** que dejó el procedimiento en el apartado anterior, lo que confirma que ambos cálculos son consistentes. La función se usa dentro del **SELECT** como cualquier función nativa, y la lógica del cálculo queda en un solo sitio.

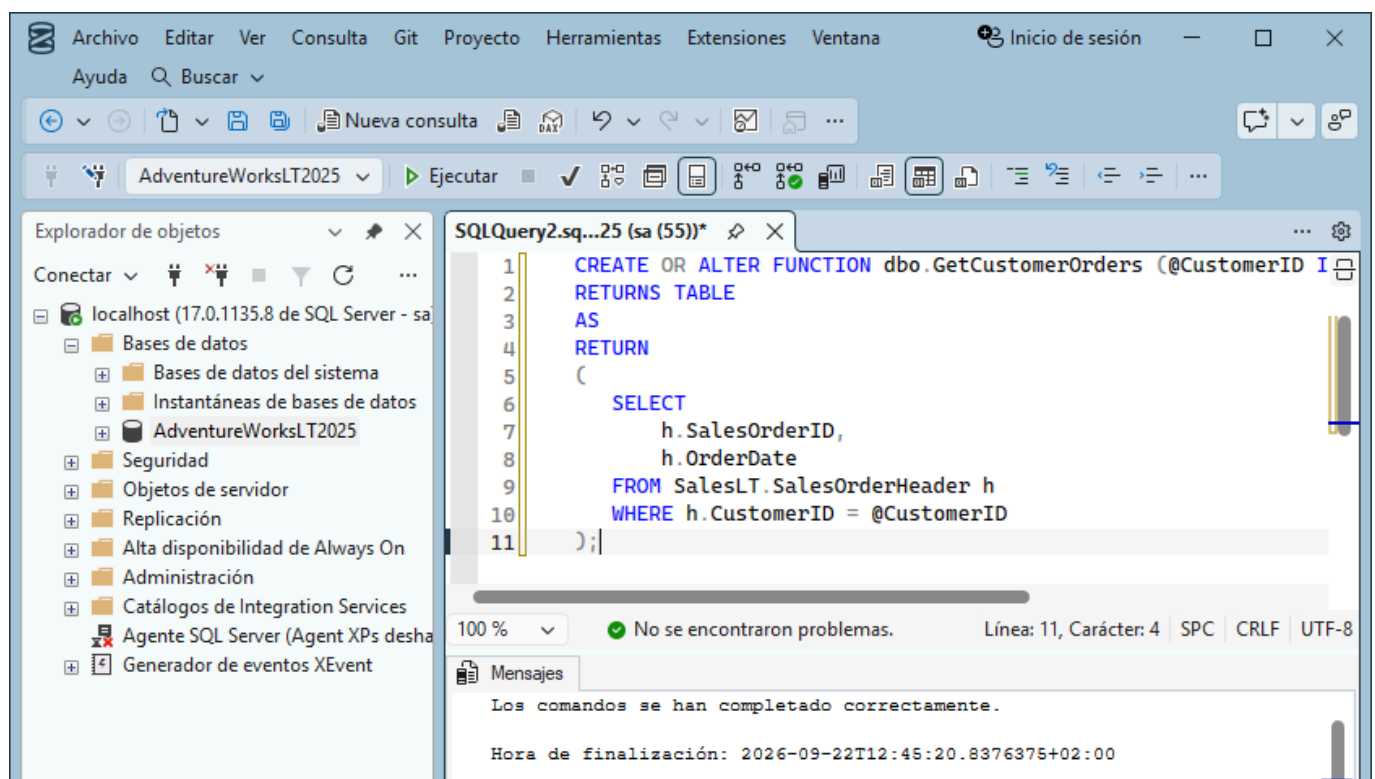
Nota: las funciones escalares se ejecutan fila a fila; en tablas grandes pueden penalizar el rendimiento frente a un **SUM** con **GROUP BY** directo.

5. Función con valores de tabla en línea (TVF)

5.1 Crear la TVF

Una TVF en línea devuelve una **tabla** a partir de un parámetro: se comporta como una "vista con parámetros".

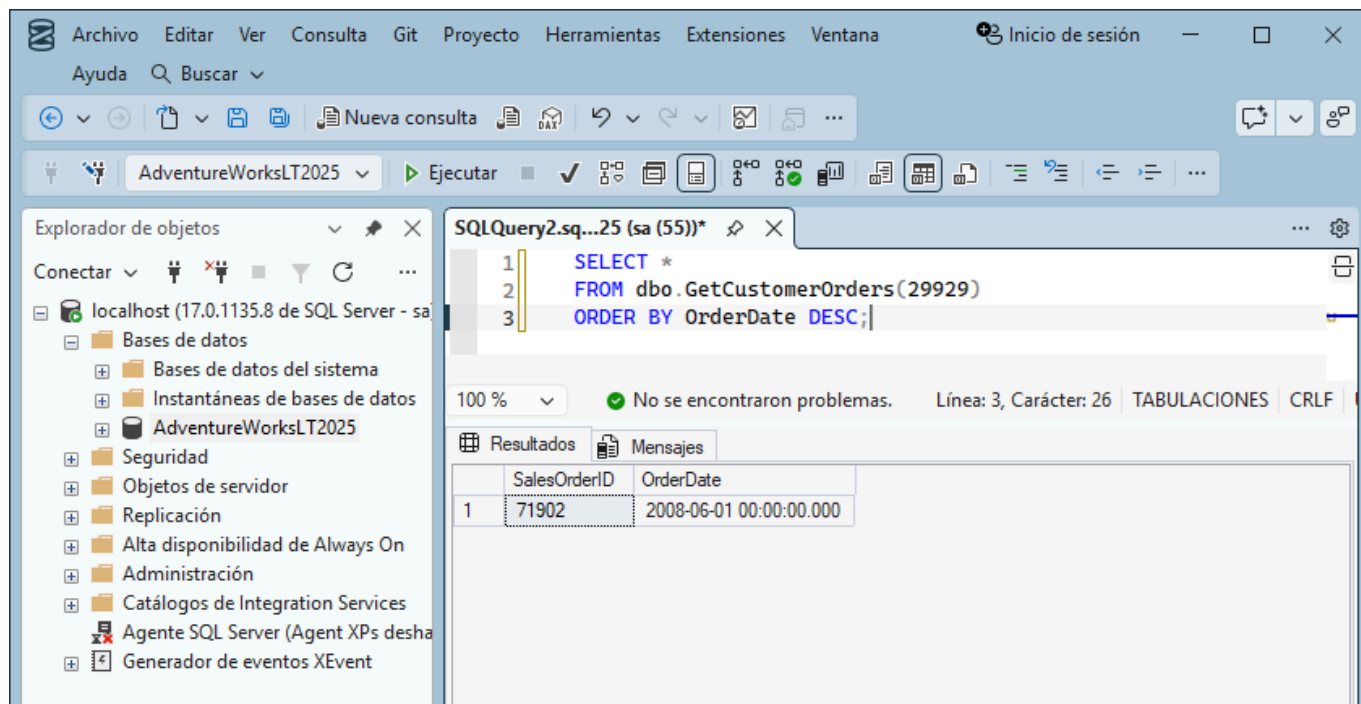
```
CREATE OR ALTER FUNCTION dbo.GetCustomerOrders (@CustomerID INT)
RETURNS TABLE
AS
RETURN
(
    SELECT h.SalesOrderID, h.OrderDate
    FROM SalesLT.SalesOrderHeader h
    WHERE h.CustomerID = @CustomerID
);
```



5.2 Consultar la TVF

Se usa en el **FROM** igual que una tabla:

```
SELECT * FROM dbo.GetCustomerOrders(29929) ORDER BY OrderDate DESC;
```

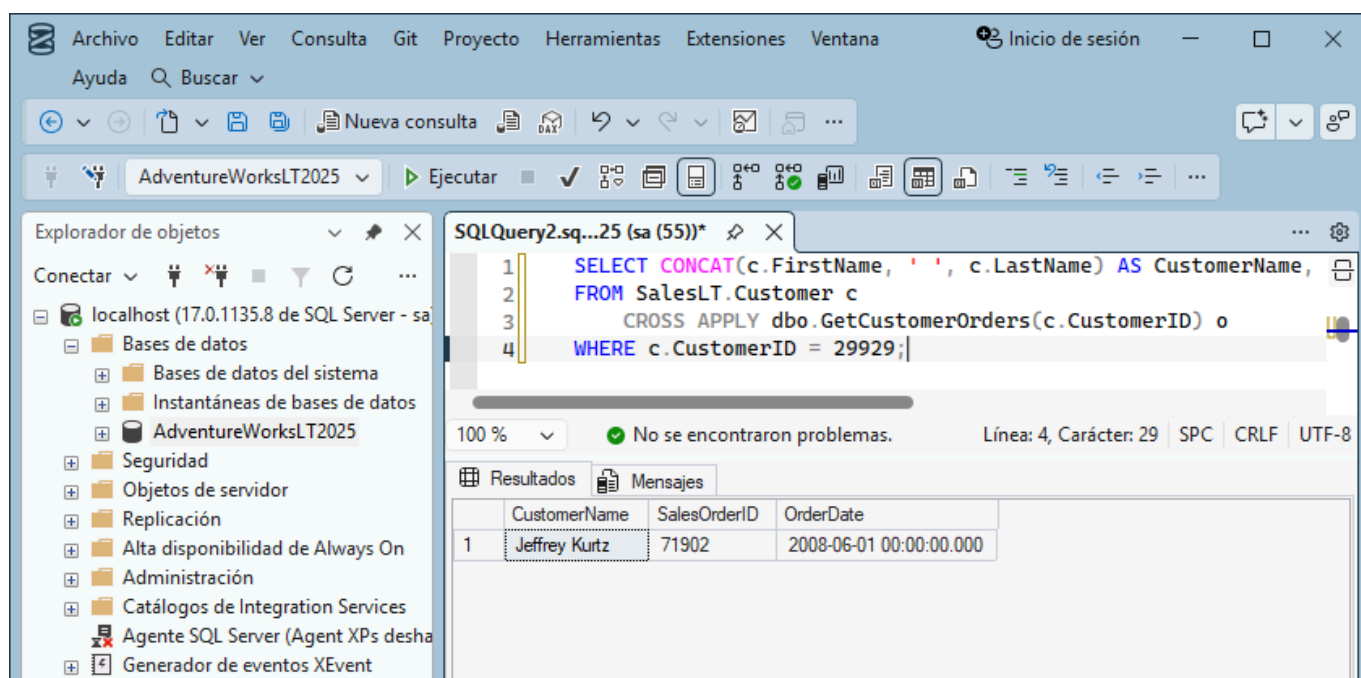


Resultado: el cliente 29929 tiene un único pedido, el 71902, con su **OrderDate**. La función devuelve una tabla de una sola fila.

5.3 Combinar la TVF con otras tablas (**CROSS APPLY**)

```
SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, o.SalesOrderID,
o.OrderDate
FROM SalesLT.Customer c
     CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
WHERE c.CustomerID = 29929;
```

CROSS APPLY ejecuta la función **por cada fila** de **Customer**, pasándole su **CustomerID**. Un **JOIN** normal no puede pasar columnas de la tabla izquierda como parámetro de una función; **APPLY** sí.



Resultado: el mismo pedido 71902, pero ahora acompañado del nombre del cliente (Jeffrey Kurtz).

6. Trigger para registrar cambios (auditoría)

6.1 Crear la tabla de auditoría y el trigger

Primero se crea `dbo.OrderAudit` (solo si no existe) y después el trigger `AFTER INSERT, UPDATE` sobre `SalesLT.SalesOrderDetail`.

```
IF OBJECT_ID('dbo.OrderAudit') IS NULL
BEGIN
    CREATE TABLE dbo.OrderAudit (
        AuditID    INT IDENTITY(1,1) PRIMARY KEY,
        OrderID    INT NOT NULL,
        OldTotal   DECIMAL(18,2) NULL,
        NewTotal   DECIMAL(18,2) NULL,
        ChangedAt  DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
    );
END
GO

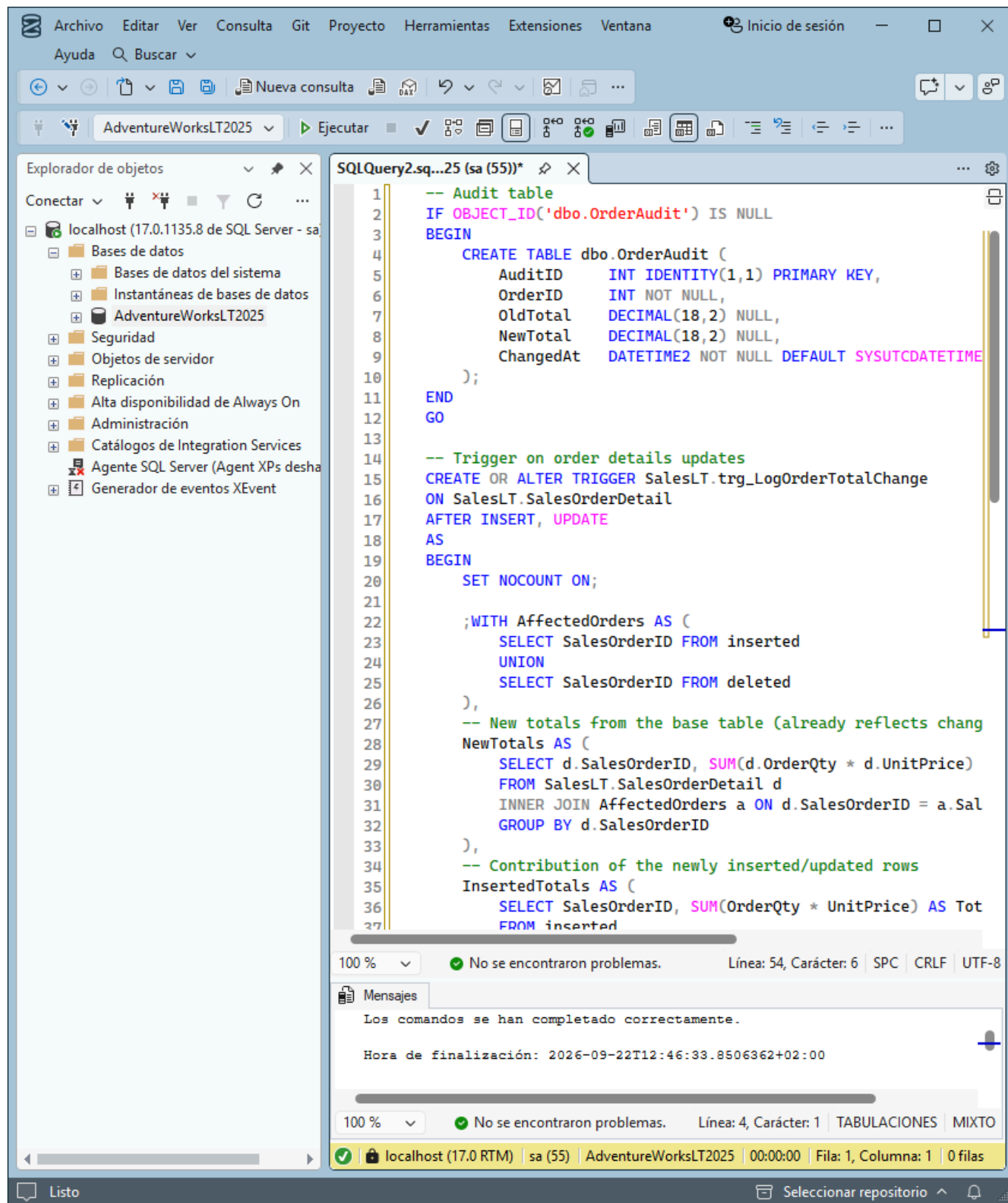
CREATE OR ALTER TRIGGER SalesLT.trg_LogOrderTotalChange
ON SalesLT.SalesOrderDetail
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    ;WITH AffectedOrders AS (
        SELECT SalesOrderID FROM inserted
        UNION
        SELECT SalesOrderID FROM deleted
    ),
    NewTotals AS (
        SELECT d.SalesOrderID, SUM(d.OrderQty * d.UnitPrice) AS Total
        FROM SalesLT.SalesOrderDetail d
        INNER JOIN AffectedOrders a ON d.SalesOrderID = a.SalesOrderID
        GROUP BY d.SalesOrderID
    ),
    InsertedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM inserted GROUP BY SalesOrderID
    ),
    DeletedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM deleted GROUP BY SalesOrderID
    )
    INSERT INTO dbo.OrderAudit (OrderID, OldTotal, NewTotal)
    SELECT
        n.SalesOrderID,
        n.Total - ISNULL(i.Total, 0) + ISNULL(d.Total, 0) AS OldTotal,
```

```
n.Total AS NewTotal
FROM NewTotals n
LEFT JOIN InsertedTotals i ON n.SalesOrderID = i.SalesOrderID
LEFT JOIN DeletedTotals d ON n.SalesOrderID = d.SalesOrderID;
END;
```

Claves para entenderlo:

- **inserted / deleted**: tablas virtuales del trigger. **inserted** tiene las filas nuevas (o su versión nueva en un **UPDATE**); **deleted** tiene la versión anterior (vacía en un **INSERT**).
- **GO** separa lotes. **CREATE TRIGGER** tiene que ser la primera instrucción de su lote, y lo mismo ocurre con **CREATE VIEW**, **CREATE FUNCTION** y **CREATE PROCEDURE**: por eso hay que cerrar el bloque de la tabla antes de empezar el del trigger.
- **CTEs (WITH)**:
 - **AffectedOrders**: pedidos afectados por el cambio.
 - **NewTotals**: total actual del pedido (la tabla base ya refleja el cambio porque el trigger es **AFTER**).
 - **InsertedTotals / DeletedTotals**: aportación de las filas nuevas y antiguas.
- **Total anterior** = total nuevo – lo que aportan las filas nuevas + lo que aportaban las antiguas.
- El trigger trabaja por **conjuntos**, así que funciona aunque una misma sentencia modifique varias filas o varios pedidos.



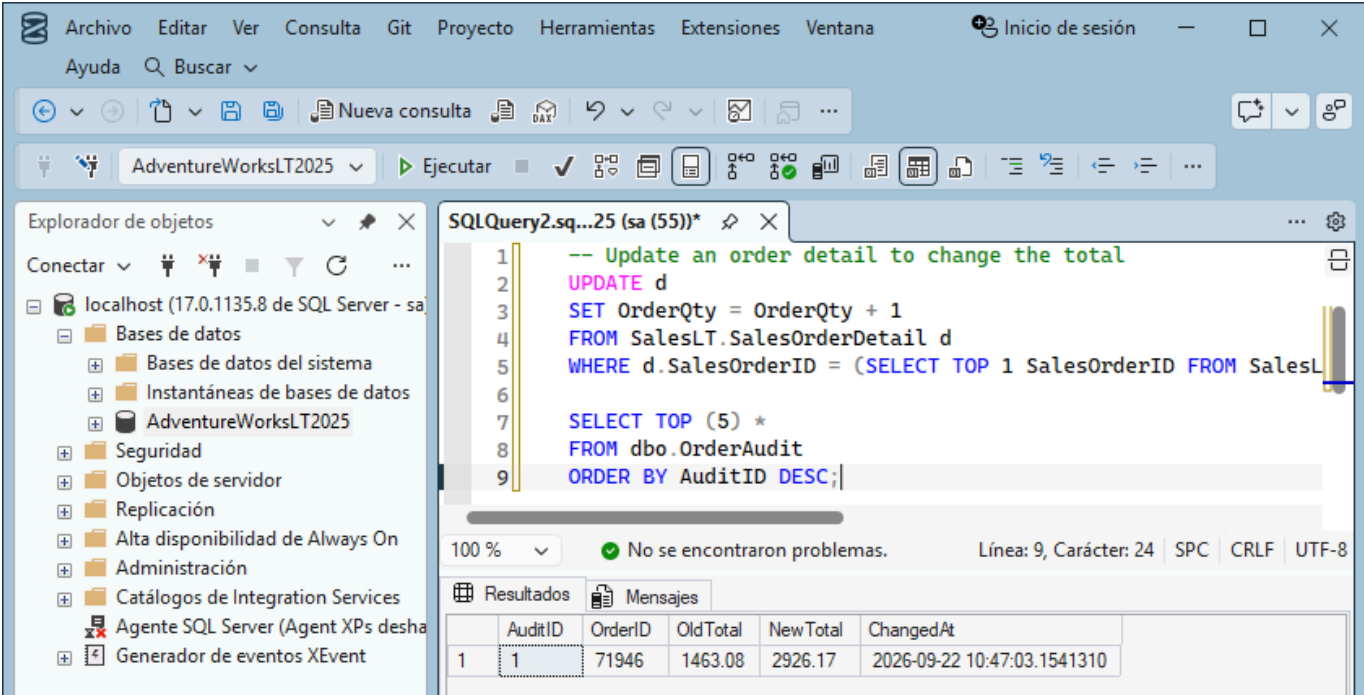
Detalle que conviene tener presente: el trigger calcula los totales como `OrderQty * UnitPrice`, mientras que el procedimiento y `fnOrderTotal` usan `LineTotal`, que es una columna calculada que además descuenta `UnitPriceDiscount`. En este laboratorio los dos valores coinciden porque ninguna línea tiene descuento, pero en un caso real con descuentos la auditoría registraría un importe distinto del total del pedido.

6.2 Probar el trigger

Se incrementa en 1 la cantidad de todas las líneas del último pedido y se consulta la auditoría:

```
UPDATE d
SET OrderQty = OrderQty + 1
FROM SalesLT.SalesOrderDetail d
WHERE d.SalesOrderID = (SELECT TOP 1 SalesOrderID FROM SalesLT.SalesOrderHeader
ORDER BY SalesOrderID DESC);

SELECT TOP (5) * FROM dbo.OrderAudit ORDER BY AuditID DESC;
```



Resultado: aparece en `dbo.OrderAudit` una fila con el pedido 71946, su `OldTotal` (1463,08), el `NewTotal` (2926,17, prácticamente el doble al haber subido en 1 la cantidad de las dos líneas) y la fecha/hora en `ChangedAt`. El registro se ha generado **automáticamente**, sin que la consulta de `UPDATE` lo pida.

La auditoría contiene una sola fila, y es lo esperado: la línea que insertó el procedimiento en el apartado 3 no aparece porque el trigger todavía no existía en ese momento. Un trigger solo reacciona a los cambios posteriores a su creación; si repitiera ahora la prueba del apartado 3, se registraría también esa inserción.

7. Limpieza (opcional)

Si no se va a volver a usar la base de datos: en SSMS → **Object Explorer** → **Databases** → clic derecho en **AdventureWorksLT2025** → **Delete** → marcar **Close existing connections** → **OK**.

Conclusiones

Objeto	Qué he aprendido
Vista	Encapsula un <code>JOIN</code> y expone los datos como una tabla virtual; no almacena datos.

Objeto	Qué he aprendido
Procedimiento	Agrupar validaciones y cambios en una transacción atómica con control de errores (<code>THROW</code> , <code>ROLLBACK</code>).
Función escalar	Centraliza un cálculo que devuelve un único valor y se usa dentro de un <code>SELECT</code> .
TVF en línea	Actúa como una vista parametrizada; se combina con otras tablas mediante <code>CROSS APPLY</code> .
Trigger	Se ejecuta solo ante <code>INSERT/UPDATE</code> y, con <code>inserted/deleted</code> , permite auditar cambios automáticamente, pero únicamente a partir del momento en que se crea.

Posibles mejoras: unificar el criterio de cálculo para que el trigger use `LineTotal` como el resto de objetos y tenga en cuenta los descuentos; ampliar el procedimiento para que recalcule también `TaxAmt` y `Freight`, y para admitir varias líneas por pedido; añadir índices sobre las columnas usadas en `JOIN` y filtros; y conceder permisos sobre la vista para exponerla de forma segura.